

CMPE344

Database Management Systems and Programming 2

Report

Smart Farming

BY

22313259 Sara Mabrouk

22304307 Abdulaziz Kulfat

22318565 Salma Saada

22312476 Mutaz Abuallan

22204025 Carissa Fredrick

Submitting Day:

17th May

1. Introduction:

This project presents the design of a Smart Farming management system. It's designed to store and organize all the data needed for modern agricultural operations. This database stores information about the User, the Farm and the Field, Crops and Harvests, Sensor and Sensor Reading and Alert, Actuator and Actuator Action, and Irrigation.

The project was modeled using an ER diagram and implemented through SQL DDL and DML statements.

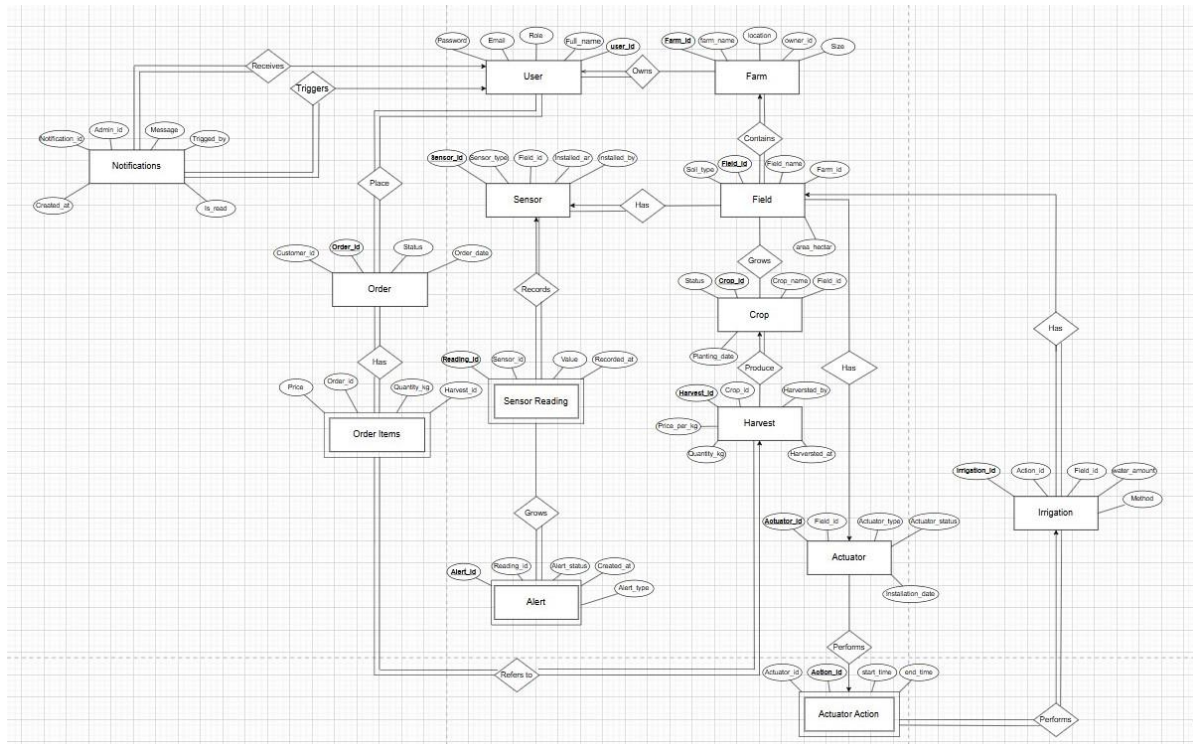
Implementing Queries, Functions and Procedures to make the system handle and data automatically.

Building a Graphical User Interface using Python to allow users to interact with the database easily.

2. Database:

a. ER Diagram:

The ER Diagram is a map of our database. It shows how the entities are linked together. It also helps us visualize the data clearly and ensures there is no repeated information.



We identified the components and represented each one as an entity:

- **User:** Represents Admin, Customer, Farmer and Technical stuff with attributes like role and contact information.
- **Farm:** Stores information about the physical farm, location, size and ownership.
- **Field:** Represents the specific areas on the farm where different crops are planted and grown.
- **Sensor:** Hardware device installed in the field.
- **Sensor Reading:** A weak entity that records time-series data values from the sensor.
- **Crop:** Information about a specific plant, including planting dates and status.
- **Harvest:** Records the yield produced from a specific crop, including quantity and date.
- **Actuator:** Physical hardware that performs mechanical actions.
- **Actuator Action:** A weak entity that records every time a device is turned on or off.
- **Irrigation:** Stores details about watering activities, including the amount of water and the duration of the process.

- **Alert:** System notifications triggered by the sensor readings to warn users of specific conditions.
- **Order / Order Items:** Manages the commercial sale of harvested produce to customers.
- **Notification:** Stores the messages sent to the user about system events. It tracks the message content.

The ER Diagram shows how entities interact:

- **User → Farm:** One user can own multiple farms (one-to-many).
- **Farm → Field:** Each farm contains multiple fields (one-to-many).
- **Field → Crop:** A field is used to grow specific crops (one-to-many).
- **Crop → Harvest:** One crop cycle can result in multiple sessions (one-to-many).
- **Field → Sensor:** A field can have multiple sensors (one-to-many).
- **Sensor → Sensor Reading:** Each sensor generates multiple data readings (one-to-many).
- **Alert → Sensor Reading:** A specific data reading triggers an alert to warn the user if something is wrong.
- **Field → Actuator:** Each field has assigned actuators to manage automations (one-to-many).
- **Actuator → Irrigation:** An actuator performs specific irrigation tasks (one-to-many).
- **Order → Order Items:** One order can consist of multiple specific harvested items (one-to-many).
- **User → Notification:** One user can receive or trigger many different notifications (one-to-many).

In this ERD, we utilized four weak entities (Sensor Reading, Actuator Action, Order Item and Alert), these entities do not have their own primary key. They depend on their parent entity to exist.

b. DML and DDL:

We have 14 tables we will present some of them:

• User table:

user_id	full_name	email	password_hash	role	created_at
1	Michael Admin	michael.admin@smartfarm.cy	admin123	admin	2026-04-26 17:04:25.727676
2	Yiannis Georgiou	yiannis.g@smartfarm.cy	farmer123	farmer	2026-04-26 17:04:25.727676
3	Maria Papadopoulou	maria.p@smartfarm.cy	farmer123	farmer	2026-04-26 17:04:25.727676
4	Andreas Christodoulou	andreas.c@smartfarm.cy	farmer123	farmer	2026-04-26 17:04:25.727676
5	Kostas Ioannou	kostas.tech@smartfarm.cy	tech123	technical_staff	2026-04-26 17:04:25.727676
6	Elena Stavrou	elena.tech@smartfarm.cy	tech123	technical_staff	2026-04-26 17:04:25.727676
7	Nikos Petrou	nikos.tech@smartfarm.cy	tech123	technical_staff	2026-04-26 17:04:25.727676
8	Sophie Laurent	sophie.l@gmail.com	customer123	customer	2026-04-26 17:04:25.727676
9	James Mitchell	james.m@outlook.com	customer123	customer	2026-04-26 17:04:25.727676
10	Fatima Al-Hassan	fatima.h@gmail.com	customer123	customer	2026-04-26 17:04:25.727676
11	Dimitris Vassiliou	dimitris.v@yahoo.com	customer123	customer	2026-04-26 17:04:25.727676
12	Anna Economou	anna.eco@gmail.com	customer123	customer	2026-04-26 17:04:25.727676
13	AbdulAziz Kulfat	22304307@student.ciu.edu.tr	123456789	admin	2026-05-03 18:14:12.690692
14	mutaz abuallan	mutaz@gmail.com	123456789	customer	2026-05-05 16:31:48.233672

```
-- 1. USERS
-- Passwords are hashed versions of 'Password123!' for all users
INSERT INTO users (full_name, email, password_hash, role) VALUES
('Michael Admin', 'michael.admin@smartfarm.cy', '$2b$12$LQv3c1yqBWHxkd0LHAKCOYz6TtxMUBpZGKM7xKZt5Rz5Fq5K5K5K', 'admin'),
('Yiannis Georgiou', 'yiannis.g@smartfarm.cy', '$2b$12$LQv3c1yqBWHxkd0LHAKCOYz6TtxMUBpZGKM7xKZt5Rz5Fq5K5K5K', 'farmer'),
('Maria Papadopoulou', 'maria.p@smartfarm.cy', '$2b$12$LQv3c1yqBWHxkd0LHAKCOYz6TtxMUBpZGKM7xKZt5Rz5Fq5K5K5K', 'farmer'),
('Andreas Christodoulou', 'andreas.c@smartfarm.cy', '$2b$12$LQv3c1yqBWHxkd0LHAKCOYz6TtxMUBpZGKM7xKZt5Rz5Fq5K5K5K', 'farmer'),
('Kostas Ioannou', 'kostas.tech@smartfarm.cy', '$2b$12$LQv3c1yqBWHxkd0LHAKCOYz6TtxMUBpZGKM7xKZt5Rz5Fq5K5K5K', 'technical_staff'),
('Elena Stavrou', 'elena.tech@smartfarm.cy', '$2b$12$LQv3c1yqBWHxkd0LHAKCOYz6TtxMUBpZGKM7xKZt5Rz5Fq5K5K5K', 'technical_staff'),
('Nikos Petrou', 'nikos.tech@smartfarm.cy', '$2b$12$LQv3c1yqBWHxkd0LHAKCOYz6TtxMUBpZGKM7xKZt5Rz5Fq5K5K5K', 'technical_staff'),
('Sophie Laurent', 'sophie.l@gmail.com', '$2b$12$LQv3c1yqBWHxkd0LHAKCOYz6TtxMUBpZGKM7xKZt5Rz5Fq5K5K5K', 'customer'),
('James Mitchell', 'james.m@outlook.com', '$2b$12$LQv3c1yqBWHxkd0LHAKCOYz6TtxMUBpZGKM7xKZt5Rz5Fq5K5K5K', 'customer'),
('Fatima Al-Hassan', 'fatima.h@gmail.com', '$2b$12$LQv3c1yqBWHxkd0LHAKCOYz6TtxMUBpZGKM7xKZt5Rz5Fq5K5K5K', 'customer'),
('Dimitris Vassiliou', 'dimitris.v@yahoo.com', '$2b$12$LQv3c1yqBWHxkd0LHAKCOYz6TtxMUBpZGKM7xKZt5Rz5Fq5K5K5K', 'customer'),
('Anna Economou', 'anna.eco@gmail.com', '$2b$12$LQv3c1yqBWHxkd0LHAKCOYz6TtxMUBpZGKM7xKZt5Rz5Fq5K5K5K', 'customer');
```

• Farm table:

farm_id	farm_name	location	size	owner_id	created_at
1	Georgiou Organic Farm	Morphou, Nicosia District, Cyprus	85.50	2	2026-04-26 17:04:25.727676
2	Papadopoulou Family Farm	Pissouri, Limassol District, Cyprus	62.75	3	2026-04-26 17:04:25.727676
3	Christodoulou Agri	Kokkinochoria, Famagusta, Cyprus	110.00	4	2026-04-26 17:04:25.727676
4	my farm	morocco	1000.00	15	2026-05-07 07:24:16.078925
5	Test Farm	Test Location	10.00	2	2026-05-07 08:16:17.051187
6	testing 1	nicosia	200.00	15	2026-05-07 08:19:03.521197

```
-- 2. FARMS
-- Realistic Cypriot farm locations with real place names
INSERT INTO farms (farm_name, location, size, owner_id) VALUES
('Georgiou Organic Farm', 'Morphou, Nicosia District, Cyprus', 85.50, 2),
('Papadopoulou Family Farm', 'Pissouri, Limassol District, Cyprus', 62.75, 3),
('Christodoulou Agri', 'Kokkinochoria, Famagusta, Cyprus', 110.00, 4);
```

● Field table:

 field_id i.. ▾	field_name varchar ▾	 farm_id i.. ▾	soil_type varchar ▾	area_hectares numeric ▾
1	North Grain Field	1 	loamy	22.00
2	South Vegetable Plot	1 	sandy	18.50
3	East Orchard Zone	1 	loamy	15.00
4	Main Citrus Field	2 	sandy	25.00
5	Vineyard Block A	2 	clay	20.75
6	Greenhouse Section	2 	silty	17.00
7	Potato Field Alpha	3 	sandy	35.00
8	Potato Field Beta	3 	sandy	30.00
9	Wheat Expansion Zone	3 	loamy	45.00
10	berries	4 	silty	200.00

```
-- 3. FIELDS
-- Each farm has multiple fields with different soil types
INSERT INTO fields (field_name, farm_id, soil_type, area_hectares) VALUES
-- Georgiou Farm fields
('North Grain Field', 1, 'loamy', 22.00),
('South Vegetable Plot', 1, 'sandy', 18.50),
('East Orchard Zone', 1, 'loamy', 15.00),
-- Papadopoulou Farm fields
('Main Citrus Field', 2, 'sandy', 25.00),
('Vineyard Block A', 2, 'clay', 20.75),
('Greenhouse Section', 2, 'silty', 17.00),
-- Christodoulou Farm fields
('Potato Field Alpha', 3, 'sandy', 35.00),
('Potato Field Beta', 3, 'sandy', 30.00),
('Wheat Expansion Zone', 3, 'loamy', 45.00);
```

● Sensors table:

 sensor_id ▾	 field_id i.. ▾	sensor_type varchar ▾	 installed_by i.. ▾	installed_at timestamp ▾
1	1 	soil_moisture	5 	2025-12-01 09:00:00
2	1 	temperature	5 	2025-12-01 09:30:00
3	2 	humidity	6 	2025-12-05 10:00:00
4	2 	soil_moisture	6 	2025-12-05 10:30:00
5	3 	temperature	5 	2025-12-10 08:00:00
6	4 	soil_moisture	7 	2026-01-05 09:00:00
7	4 	humidity	7 	2026-01-05 09:30:00
8	5 	temperature	6 	2026-01-10 10:00:00
9	7 	soil_moisture	5 	2025-10-15 08:00:00
10	7 	temperature	5 	2025-10-15 08:30:00
11	8 	humidity	7 	2025-10-20 09:00:00
12	9 	soil_moisture	6 	2025-11-01 10:00:00
13	10 	soil_moisture	16 	2026-05-10 13:22:08.603629

```
-- 8. SENSORS
-- Installed by technical staff members
INSERT INTO sensors (field_id, sensor_type, installed_by, installed_at) VALUES
(1, 'soil_moisture', 5, '2025-12-01 09:00:00'),
(1, 'temperature', 5, '2025-12-01 09:30:00'),
(2, 'humidity', 6, '2025-12-05 10:00:00'),
(2, 'soil_moisture', 6, '2025-12-05 10:30:00'),
(3, 'temperature', 5, '2025-12-10 08:00:00'),
(4, 'soil_moisture', 7, '2026-01-05 09:00:00'),
(4, 'humidity', 7, '2026-01-05 09:30:00'),
(5, 'temperature', 6, '2026-01-10 10:00:00'),
(7, 'soil_moisture', 5, '2025-10-15 08:00:00'),
(7, 'temperature', 5, '2025-10-15 08:30:00'),
(8, 'humidity', 7, '2025-10-20 09:00:00'),
(9, 'soil_moisture', 6, '2025-11-01 10:00:00');
```

● Actuators table:

actuator_id i	field_id i..	actuator_type varchar	actuator_status varchar	installation_date date
1	1 →	Drip Irrigation Pump	active	2025-12-01
2	2 →	Sprinkler System	active	2025-12-05
3	3 →	Drip Irrigation Pump	active	2025-12-10
4	4 →	Flood Irrigation Gate	active	2026-01-05
5	5 →	Sprinkler System	maintenance	2026-01-10
6	7 →	Drip Irrigation Pump	active	2025-10-15
7	8 →	Sprinkler System	active	2025-10-20
8	9 →	Flood Irrigation Gate	inactive	2025-11-01
9	10 →	soil	active	2026-05-10

```
-- 11. ACTUATORS
INSERT INTO actuators (field_id, actuator_type, actuator_status, installation_date) VALUES
(1, 'Drip Irrigation Pump', 'active', '2025-12-01'),
(2, 'Sprinkler System', 'active', '2025-12-05'),
(3, 'Drip Irrigation Pump', 'active', '2025-12-10'),
(4, 'Flood Irrigation Gate', 'active', '2026-01-05'),
(5, 'Sprinkler System', 'maintenance', '2026-01-10'),
(7, 'Drip Irrigation Pump', 'active', '2025-10-15'),
(8, 'Sprinkler System', 'active', '2025-10-20'),
(9, 'Flood Irrigation Gate', 'inactive', '2025-11-01');
```

● Alert table:

alert_id i.	field_id i..	reading_id	alert_type varchar	alert_status varchar	created_at timestamp
1	1 →	3 →	Low Soil Moisture - Below 30%	resolved	2026-04-22 06:05:00
2	1 →	7 →	High Temperature - Above 36°C	resolved	2026-04-22 06:05:00
3	2 →	11 →	High Humidity - Above 78%	resolved	2026-04-22 07:05:00
4	4 →	18 →	Low Soil Moisture - Below 30%	resolved	2026-04-22 08:05:00
5	7 →	24 →	High Temperature - Above 37°C	resolved	2026-04-22 06:35:00
6	3 →	NULL	Pest Detection - Aphids spotted	resolved	2026-04-21 09:00:00
7	5 →	NULL	Equipment Malfunction - Sensor down	resolved	2026-04-18 14:00:00
8	9 →	NULL	Weed Infestation Reported	resolved	2026-04-20 11:00:00


```
-- 10. ALERTS
-- Mix of automatic (from sensor readings) and manual alerts
INSERT INTO alerts (field_id, reading_id, alert_type, alert_status, created_at) VALUES
-- Automatic alerts (triggered by sensor readings)
(1, 3, 'Low Soil Moisture - Below 30%', 'resolved', '2026-04-22 06:05:00'),
(1, 7, 'High Temperature - Above 36°C', 'resolved', '2026-04-22 06:05:00'),
(2, 11, 'High Humidity - Above 78%', 'active', '2026-04-22 07:05:00'),
(4, 18, 'Low Soil Moisture - Below 30%', 'active', '2026-04-22 08:05:00'),
(7, 24, 'High Temperature - Above 37°C', 'active', '2026-04-22 06:35:00'),
-- Manual alerts (no reading_id, field_id tells us where)
(3, NULL, 'Pest Detection - Aphids spotted', 'active', '2026-04-21 09:00:00'),
(5, NULL, 'Equipment Malfunction - Sensor down', 'resolved', '2026-04-18 14:00:00'),
(9, NULL, 'Weed Infestation Reported', 'active', '2026-04-20 11:00:00');
```

• Crops table:

crop_id i..	crop_name varchar	field_id i..	status varchar	planting_date date
1	Winter Wheat	1 →	growing	2026-01-15
2	Cherry Tomatoes	2 →	growing	2026-02-20
3	Valencia Oranges	3 →	harvested	2025-03-10
4	Navel Oranges	4 →	harvested	2025-04-01
5	Cabernet Sauvignon	5 →	growing	2025-10-15
6	Bell Peppers	6 →	growing	2026-02-01
7	Spunta Potatoes	7 →	harvested	2025-11-01
8	Cara Potatoes	8 →	growing	2026-01-10
9	Durum Wheat	9 →	growing	2025-12-20
10	berry	10 →	harvested	2026-04-01
11	Tomato	10 →	harvested	2026-05-10
12	Tomato	10 →	growing	2026-05-10

```
-- 4. CROPS
INSERT INTO crops (crop_name, field_id, status, planting_date) VALUES
-- Georgiou Farm crops
('Winter Wheat', 1, 'growing', '2026-01-15'),
('Cherry Tomatoes', 2, 'growing', '2026-02-20'),
('Valencia Oranges', 3, 'harvested', '2025-03-10'),
-- Papadopoulou Farm crops
('Navel Oranges', 4, 'harvested', '2025-04-01'),
('Cabernet Sauvignon', 5, 'growing', '2025-10-15'),
('Bell Peppers', 6, 'growing', '2026-02-01'),
-- Christodoulou Farm crops
('Spunta Potatoes', 7, 'harvested', '2025-11-01'),
('Cara Potatoes', 8, 'growing', '2026-01-10'),
('Durum Wheat', 9, 'growing', '2025-12-20');
```

c. Roles:

```
-- STEP 1: Create the roles
CREATE ROLE admin_role;
CREATE ROLE farmer_role;
CREATE ROLE technical_role;
CREATE ROLE customer_role;
```

To ensure security and access control, the database defines four roles that correspond directly to the user types in the system, each role is granted only the privileges it needs to perform its responsibilities.

The user's type are:

- **Admin role (admin_role):** The admin_role is granted full privileges on every table and sequence in the public schema. It allows the administrators to read, insert, delete and manage any data in the system without restrictions.

```
-- STEP 2: ADMIN ROLE
-- Full access to everything
-- =====
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO admin_role;
GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA public TO admin_role;
```

- **Farmer role (farmer_role):** The farmer_role has access to the farming tables: farms, fields, crops and harvests. This allows farmers to manage their own agriculture data. They are granted read-only access on the monitoring tables such as sensors, sensor_readings, alerts, actuators, actuator_actions and irrigations.

```
-- Manages their own farms, fields, crops and harvests
-- Can view sensors and alerts for their fields
-- =====

-- Full access to farming tables
GRANT SELECT, INSERT, UPDATE, DELETE ON farms TO farmer_role;
GRANT SELECT, INSERT, UPDATE, DELETE ON fields TO farmer_role;
GRANT SELECT, INSERT, UPDATE, DELETE ON crops TO farmer_role;
GRANT SELECT, INSERT, UPDATE, DELETE ON harvests TO farmer_role;

-- Read only on monitoring tables
GRANT SELECT ON sensors TO farmer_role;
GRANT SELECT ON sensor_readings TO farmer_role;
GRANT SELECT ON alerts TO farmer_role;
GRANT SELECT ON actuators TO farmer_role;
GRANT SELECT ON actuator_actions TO farmer_role;

GRANT SELECT ON irrigation TO farmer_role;

-- No access to other users data
GRANT SELECT ON users TO farmer_role;

-- Allow sequences for INSERT operations
GRANT USAGE, SELECT ON SEQUENCE farms_farm_id_seq TO farmer_role;
GRANT USAGE, SELECT ON SEQUENCE fields_field_id_seq TO farmer_role;
GRANT USAGE, SELECT ON SEQUENCE crops_crop_id_seq TO farmer_role;
GRANT USAGE, SELECT ON SEQUENCE harvests_harvest_id_seq TO farmer_role;
```

- **Technical Staff role (technical_role):** The technical_role is designed for staff who manages the physical infrastructure of the farm. They have full access on the technical tables like sensors, sensor_readings, alerts, actuators, actuator_actions and irrigation. They can only view the farms, fields, crops and users tables to understand the farm structure but they cannot modify it.

```

-- Read only on farm structure
GRANT SELECT ON farms TO technical_role;
GRANT SELECT ON fields TO technical_role;
GRANT SELECT ON crops TO technical_role;
GRANT SELECT ON users TO technical_role;

-- Full access to technical tables
GRANT SELECT, INSERT, UPDATE, DELETE ON sensors TO technical_role;
GRANT SELECT, INSERT, UPDATE, DELETE ON sensor_readings TO technical_role;
GRANT SELECT, INSERT, UPDATE, DELETE ON alerts TO technical_role;
GRANT SELECT, INSERT, UPDATE, DELETE ON actuators TO technical_role;
GRANT SELECT, INSERT, UPDATE, DELETE ON actuator_actions TO technical_role;
GRANT SELECT, INSERT, UPDATE, DELETE ON irrigation TO technical_role;

-- No access to orders or harvests
REVOKE ALL ON orders FROM technical_role;
REVOKE ALL ON order_items FROM technical_role;
REVOKE ALL ON harvests FROM technical_role;

-- Allow sequences for INSERT operations
GRANT USAGE, SELECT ON SEQUENCE sensors_sensor_id_seq TO technical_role;
GRANT USAGE, SELECT ON SEQUENCE sensor_readings_reading_id_seq TO technical_role;
GRANT USAGE, SELECT ON SEQUENCE alerts_alert_id_seq TO technical_role;
GRANT USAGE, SELECT ON SEQUENCE actuators_actuator_id_seq TO technical_role;
GRANT USAGE, SELECT ON SEQUENCE actuator_actions_action_id_seq TO technical_role;
GRANT USAGE, SELECT ON SEQUENCE irrigation_irrigation_id_seq TO technical_role;

```

- **Customer role (customer_role):** The customer_role gives customers the ability to browse available produce and manage their own purchases. They have Select access on farms, fields, crops and harvests so they can view what products are available. They have the full access to the orders, order_items tables so they can place orders and track them.

```

-- Read only on farming info
GRANT SELECT ON farms TO customer_role;
GRANT SELECT ON fields TO customer_role;
GRANT SELECT ON crops TO customer_role;
GRANT SELECT ON harvests TO customer_role;

-- Full access to orders
GRANT SELECT, INSERT, UPDATE ON orders TO customer_role;
GRANT SELECT, INSERT, UPDATE ON order_items TO customer_role;

-- No access to sensors, actuators, alerts or other users
REVOKE ALL ON sensors FROM customer_role;
REVOKE ALL ON sensor_readings FROM customer_role;
REVOKE ALL ON alerts FROM customer_role;
REVOKE ALL ON actuators FROM customer_role;
REVOKE ALL ON actuator_actions FROM customer_role;

REVOKE ALL ON irrigation FROM customer_role;

-- Allow sequences for INSERT operations
GRANT USAGE, SELECT ON SEQUENCE orders_order_id_seq TO customer_role;
GRANT USAGE, SELECT ON SEQUENCE order_items_order_item_id_seq TO customer_role;

```

d. Queries, Functions, Procedures, Views and Triggers:

Procedure: add_harvest

```
CREATE OR REPLACE PROCEDURE add_harvest(  
    p_crop_id      INT,  
    p_harvested_by INT,  
    p_harvest_date TIMESTAMP,  
    p_quantity_kg  DECIMAL  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    INSERT INTO harvests (crop_id, harvested_by, quantity_kg, harvested_at)  
    VALUES (p_crop_id, p_harvested_by, p_quantity_kg, p_harvest_date);  
  
    UPDATE crops  
    SET status = 'harvested'  
    WHERE crop_id = p_crop_id;  
  
    RAISE NOTICE 'Harvest added for crop ID %', p_crop_id;  
END;  
$;
```

Explanation: This procedure records a new harvest entry by inserting the data into harvest table. Then automatically it updates the crop table to change the status that specific crop to “harvested”.

GUI Implementation:

```
supabase.rpc("add_harvest", {  
    "p_crop_id": cid,  
    "p_harvested_by": farmer_id,  
    "p_harvest_date": str(datetime.combine(hdt, datetime.min.time())),  
    "p_quantity_kg": hqty,  
}).execute()  
  
# Update price_per_kg on the newly created harvest  
latest = supabase.table("harvests")\  
    .select("harvest_id")\  
    .eq("crop_id", cid)\  
    .eq("harvested_by", farmer_id)\  
    .order("harvested_at", desc=True)\  
    .execute().data
```

Activate Windows
Go to Settings to activate Windows.

Procedure: sp_place_order (p_customer_id NUMBER)

Procedure: sp_create_alert (p_reading_id NUMBER, p_type VARCHAR2)

```

--2/ Creates a new order for a customer
CREATE PROCEDURE sp_place_order(p_customer_id NUMBER)
AS
BEGIN
    INSERT INTO orders(customer_id, status, order_date)
    VALUES (p_customer_id, 'pending', SYSDATE);

    COMMIT;
END;

--3/ Manually inserts an alert into system monitoring table
CREATE PROCEDURE sp_create_alert(p_reading_id NUMBER, p_type VARCHAR2)
AS
BEGIN
    INSERT INTO alerts(reading_id, alert_type, alert_status)
    VALUES (p_reading_id, p_type, 'active');
END;

```

Explanation:

- **sp_place_order:** This procedure automates the creation of a new customer order with a default “pending” status. It uses the current system date to ensure the order.
- **sp_create_order:** It allows the manual creation of a monitoring alert linked to a specific sensor reading. It sets the alert status to “active” so it appears immediately on the system dashboard for the user.

GUI Implementation:

```

# PROCEDURE: sp_place_order(p_customer_id)
# Creates the order record via stored procedure
supabase.rpc("sp_place_order", {
    "p_customer_id": user["user_id"]
}).execute()

```

Procedure: cancel_customer_orders (p_customer_id INT)

```
-- PROCEDURE: Cancel all pending orders for a customer
CREATE OR REPLACE PROCEDURE cancel_customer_orders(
    p_customer_id INT
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_count INT;
BEGIN
    SELECT COUNT(*) INTO v_count
    FROM orders
    WHERE customer_id = p_customer_id
    AND status = 'pending';

    IF v_count = 0 THEN
        RAISE NOTICE 'No pending orders for customer ID %', p_customer_id;
    ELSE
        UPDATE orders
        SET status = 'cancelled'
        WHERE customer_id = p_customer_id
        AND status = 'pending';

        RAISE NOTICE '% orders cancelled for customer ID %', v_count, p_customer_id;
    END IF;
END;
$$;
```

Explanation: This procedure finds and cancels all “pending” orders for a specific customer in a single action. It sends a notification to the administrator confirming how many orders were updated.

GUI Implementation:

```
# Calls the stored procedure cancel_customer_orders()
supabase.rpc("cancel_customer_orders", {
    "p_customer_id": user["user_id"]
}).execute()
st.success("✅ All pending orders cancelled.")
st.rerun()
```

Function: get_active_alerts_counts()

```
CREATE OR REPLACE FUNCTION get_active_alerts_count()
RETURNS INT
LANGUAGE plpgsql
AS $$
DECLARE
    v_count INT;
BEGIN
    SELECT COUNT(*) INTO v_count
    FROM alerts
    WHERE alert_status = 'active';

    RETURN v_count;
END;
$$;
```

Explanation: This function counts all alerts in the database that are currently marked with an “active” status. It returns this total as an integer to show the number of the urgent issues to the user interface.

GUI Implementation:

```
# FUNCTION: get_active_alerts_count() - total count of active alerts
active_count = supabase.rpc("get_active_alerts_count").execute().data or 0
st.write(f"**{len(alerts_full)} alert(s) shown** · 🟡 {active_count} active total (via `get_active_alerts_count()`)"
```

Function: get_farm_total_harvest(p_farm_id INT)

```
CREATE OR REPLACE FUNCTION get_farm_total_harvest(
    p_farm_id INT
)
RETURNS DECIMAL
LANGUAGE plpgsql
AS $$
DECLARE
    v_total DECIMAL := 0;
BEGIN
    SELECT COALESCE(SUM(h.quantity_kg), 0)
    INTO v_total
    FROM harvests h
    JOIN crops c ON h.crop_id = c.crop_id
    JOIN fields fi ON c.field_id = fi.field_id
    WHERE fi.farm_id = p_farm_id;

    RETURN v_total;
END;
$$;
```

Explanation: This function joins the harvest, crop and the field tables to calculate the total weight of all produce harvested from a specific farm.

GUI Implementation:

```
# FUNCTION: get_farm_total_harvest(p_farm_id)
# Calls the PL/SQL function to calculate total harvested kg for this farm
total_harvest_kg = supabase.rpc("get_farm_total_harvest", {"p_farm_id": f["farm_id"]}).execute().data or 0
```

Trigger: notify_admin_harvest ()

```
-- TRIGGER 2: Notify admin when farmer adds a harvest
CREATE OR REPLACE FUNCTION notify_admin_harvest()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_farmer_name    VARCHAR;
    v_admin_id       INT;
    v_crop_name      VARCHAR;
BEGIN
    SELECT full_name INTO v_farmer_name
    FROM users WHERE user_id = NEW.harvested_by;

    SELECT crop_name INTO v_crop_name
    FROM crops WHERE crop_id = NEW.crop_id;
```

```
    FOR v_admin_id IN
        SELECT user_id FROM users WHERE role = 'admin'
    LOOP
        INSERT INTO notifications (admin_id, triggered_by, message)
        VALUES (
            v_admin_id,
            NEW.harvested_by,
            '🌱 Farmer ' || v_farmer_name || ' recorded a new harvest of ' ||
            NEW.quantity_kg || 'kg of ' || v_crop_name
        );
    END LOOP;

    RETURN NEW;
END;
$;
```

```
CREATE OR REPLACE TRIGGER trg_notify_harvest
AFTER INSERT ON harvests
FOR EACH ROW
EXECUTE FUNCTION notify_admin_harvest();
```

Explanation: This trigger automatically runs whenever a farmer adds a new entry into the harvests table to alert system managers. It uses a loop to find all system administrator and inserts a custom notification message with the farmer's name, crop name, and harvest weight into the notification table.

GUI Implementation:

```
# FUNCTION: get_farm_total_harvest(p_farm_id)
with st.expander('🌾 Farm Harvest Totals - via get_farm_total_harvest() function'):
    st.caption("Calls PL/SQL function get_farm_total_harvest(p_farm_id) for each farm")
    for farm in farms:
        # FUNCTION: get_farm_total_harvest(p_farm_id) - returns total harvested kg
        result = supabase.rpc("get_farm_total_harvest", {"p_farm_id": farm["farm_id"]}).execute().data
        total = float(result) if result is not None else 0
        st.markdown(
            f'<div style="display:flex;justify-content:space-between;background:#262b36;border:1px solid #2a2f3a;">
            f'<border-radius:10px;padding:14px 20px;margin-bottom:10px">
            f'<span style="color:#2e8b57;font-weight:600">🌾 {farm["farm_name"]}</span>
            f'<span style="color:#3b82f6;font-weight:800">[total,.2f] kg</span></div>',
            unsafe_allow_html=True
        )
    )
```

Activate Windows

Trigger: notify_admin_order ()

```
-- TRIGGER 3: Notify admin when customer places an order
CREATE OR REPLACE FUNCTION notify_admin_order()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_customer_name VARCHAR;
    v_admin_id INT;
BEGIN
    SELECT full_name INTO v_customer_name
    FROM users WHERE user_id = NEW.customer_id;

    FOR v_admin_id IN
        SELECT user_id FROM users WHERE role = 'admin'
    LOOP
        INSERT INTO notifications (admin_id, triggered_by, message)

```

```
VALUES (
    v_admin_id,
    NEW.customer_id,
    '👤 Customer ' || v_customer_name || ' placed a new order on ' ||
    NEW.order_date
);
END LOOP;

RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_notify_order
AFTER INSERT ON orders

FOR EACH ROW
EXECUTE FUNCTION notify_admin_order();
```

Explanation: This trigger fires automatically whenever a new row is added to the orders table to keep administrators informed about commercial activity. It loops through all system administrator and inserts a tailored notification into the notification table containing the customer's name and the creation date of the order.

GUI Implementation:

```
# TRIGGER: trg_notify_order fires automatically on INSERT

# Fetch the newly created order ID
new_order = supabase.table("orders")\
    .select("order_id")\
    .eq("customer_id", user["user_id"])\
    .eq("status", "pending")\
    .order("order_id", desc=True)\
    .execute().data
```

Trigger: notify_admin_new_user ()

```
-- TRIGGER: Notify admin when new user is created
CREATE OR REPLACE FUNCTION notify_admin_new_user()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE v_admin_id INT;
BEGIN
    FOR v_admin_id IN SELECT user_id FROM users WHERE role = 'admin' LOOP
        INSERT INTO notifications (admin_id, triggered_by, message)
        VALUES (v_admin_id, NEW.user_id,
            '👤 New user registered: ' || NEW.full_name || ' (' || NEW.role || ')');
    END LOOP;
    RETURN NEW;
END;
$$;
CREATE OR REPLACE TRIGGER trg_notify_new_user
AFTER INSERT ON users FOR EACH ROW EXECUTE FUNCTION notify_admin_new_user();
```

Explanation: This trigger executes automatically whenever a new row is added to the users table to track system sign-ups.

GUI Implementation:

```
supabase.table("users").insert({
  "full_name":    new_name,
  "email":        new_email,
  "password_hash": new_pass,
  "role":         new_role,
}).execute()
# TRIGGER: trg_notify_new_user fires automatically on INSERT
st.success(f"✅ User '{new_name}' created.")
st.rerun()
except Exception as e:
  st.error(f"Error: {e}")
```

View: vw_farm_production

```
-- View 1: Farm production summary
CREATE OR REPLACE VIEW vw_farm_production AS
SELECT
  f.farm_name,
  SUM(h.quantity_kg) AS total_harvest
FROM farms f
JOIN fields fi ON f.farm_id = fi.farm_id
JOIN crops c ON fi.field_id = c.field_id
JOIN harvests h ON c.crop_id = h.crop_id
GROUP BY f.farm_name;
```

Explanation: This view joins the farms, fields, crops and harvests tables to aggregate total crop yields by farm name.

GUI Implementation:

```
# VIEW: vw_farm_production - total harvest per farm
prod_data = supabase.table("vw_farm_production").select("*").execute().data or []
if prod_data:
    df = pd.DataFrame(prod_data)
    df = df.sort_values("total_harvest", ascending=False).reset_index(drop=True)
    df.index += 1
    df.columns = ["Farm Name", "Total Harvest (kg)"]
    st.dataframe(df, use_container_width=True)
else:
    st.info("No production data yet.")
```

View: vw_active_alerts

```
-- View 2: Active alerts only
CREATE OR REPLACE VIEW vw_active_alerts AS
SELECT * FROM alerts
WHERE alert_status = 'active';

-- View 3: Irrigation summary per field
CREATE OR REPLACE VIEW vw_irrigation_summary AS
SELECT
  field_id,
  SUM(water_amount) AS total_water_used,
  COUNT(*) AS irrigation_sessions
FROM irrigation
GROUP BY field_id;
```

Explanation: This view filters the main alerts table to display only the records where the status is currently set to “active”.

GUI Implementation:

```
alert_status_filter = st.selectbox("Filter by Status", ["All", "active", "resolved", "ignored"], key="alert_filter")

if alert_status_filter == "active":
    # VIEW: vw_active_alerts - shows only active alerts for real-time monitoring
    alerts_full = supabase.table("vw_active_alerts").select("*").execute().data or []
    st.caption("🔄 Fetched via **vw_active_alerts** view")
else:
    alerts_full = supabase.table("alerts")\
        .select("*, field:fields(field_name, farm_id, farms(farm_name))")\
        .order("created_at", desc=True)\
        .execute().data or []
    if alert_status_filter != "All":
        alerts_full = [a for a in alerts_full if a["alert_status"] == alert_status_filter]
```

Query: Average sensor readings per field per sensor type

```
-- QUERY 4: Average sensor readings per field per sensor type
-- Used in: Farmer + Technical Dashboard

SELECT
    fi.field_name,
    fa.farm_name,
    s.sensor_type,
    ROUND(AVG(sr.value)::NUMERIC, 2) AS avg_reading,
    ROUND(MIN(sr.value)::NUMERIC, 2) AS min_reading,
    ROUND(MAX(sr.value)::NUMERIC, 2) AS max_reading,
    COUNT(sr.reading_id) AS total_readings
FROM sensor_readings sr
JOIN sensors s ON sr.sensor_id = s.sensor_id
JOIN fields fi ON s.field_id = fi.field_id
JOIN farms fa ON fi.farm_id = fa.farm_id
GROUP BY fi.field_name, fa.farm_name, s.sensor_type
ORDER BY fi.field_name, s.sensor_type;
```

Explanation: This query joins sensor, sensor readings, fields and farms tables to calculate average, maximum and minimum values for different sensor types across each fields.

Output:

field_name	farm_name	sensor_type	avg_reading	min_reading	max_reading	total_reading
Main Citrus Field	Papadopoulou Family Farm	soil_moisture	33.83	27.80	38.50	3
North Grain Field	Georgiou Organic Farm	soil_moisture	39.65	28.50	45.20	4
North Grain Field	Georgiou Organic Farm	temperature	24.68	18.50	36.80	4
Potato Field Alpha	Christodoulou Agri	soil_moisture	53.20	51.20	55.00	3
Potato Field Alpha	Christodoulou Agri	temperature	26.27	19.20	37.10	3
South Vegetable Plot	Georgiou Organic Farm	humidity	66.45	60.10	78.20	4
South Vegetable Plot	Georgiou Organic Farm	soil_moisture	50.67	48.90	52.30	3

Query: Total harvest per farm with farmer name

```
-- QUERY 5: Total harvest per farm with farmer name
-- Used in: Admin + Farmer Dashboard
SELECT
    fa.farm_name,
    u.full_name AS farmer_name,
    COUNT(h.harvest_id) AS total_harvests,
    SUM(h.quantity_kg) AS total_quantity_kg,
    ROUND(AVG(h.quantity_kg)::NUMERIC, 2) AS avg_harvest_kg
FROM harvests h
JOIN crops c ON h.crop_id = c.crop_id
JOIN fields fi ON c.field_id = fi.field_id
JOIN farms fa ON fi.farm_id = fa.farm_id
JOIN users u ON fa.owner_id = u.user_id
GROUP BY fa.farm_name, u.full_name
ORDER BY total_quantity_kg DESC;
```

Explanation: This query combines records from harvests, crops, fields, farms and users tables to calculate total production lined directly to each property owner.

Output:

farm_name	farmer_name	total_harvest	total_quantit	avg_harvest_k
Christodoulou Agri	Andreas Christodoulou	2	49500.00	24750.00
Georgiou Organic Farm	Yiannis Georgiou	2	22700.00	11350.00
Papadopoulou Family Farm	Maria Papadopoulou	1	22000.00	22000.00
my farm	sara mabrouk	2	14.20	7.10

Query: All orders with customer and crop details

```
SELECT
    o.order_id,
    u.full_name AS customer_name,
    c.crop_name,
    fa.farm_name,
    oi.quantity_kg,
    oi.price,
    o.status AS order_status,
    o.order_date
FROM orders o
JOIN users u ON o.customer_id = u.user_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN harvests h ON oi.harvest_id = h.harvest_id
JOIN crops c ON h.crop_id = c.crop_id
JOIN fields fi ON c.field_id = fi.field_id
JOIN farms fa ON fi.farm_id = fa.farm_id
ORDER BY o.order_date DESC;
```

Explanation: This query performs multiple sequential joins across the orders, users, order_items, harvests, crops, fields and farms tables to construct a complete transactional profile.

Output:

order_id	customer_name	crop_name	farm_name	quantity_kg	price	order_status	order_date
8	mutaz abuallan	berry	my farm	6.00	2.70	delivered	2026-05-10
6	Sophie Laurent	Spunta Potatoes	Christodoulou Agri	800.00	480.00	pending	2026-04-10
4	Dimitris Vassiliou	Navel Oranges	Papadopoulou Family Farm	400.00	320.00	pending	2026-04-05
3	Fatima Al-Hassan	Valencia Oranges	Georgiou Organic Farm	200.00	160.00	pending	2026-04-01
3	Fatima Al-Hassan	Spunta Potatoes	Christodoulou Agri	500.00	300.00	pending	2026-04-01
7	James Mitchell	Spunta Potatoes	Christodoulou Agri	600.00	360.00	delivered	2026-03-25
5	Anna Economou	Valencia Oranges	Georgiou Organic Farm	150.00	105.00	cancelled	2026-03-20
2	James Mitchell	Spunta Potatoes	Christodoulou Agri	1000.00	600.00	delivered	2026-03-12
1	Sophie Laurent	Valencia Oranges	Georgiou Organic Farm	500.00	350.00	delivered	2026-03-10

Query: Fields with no sensors installed

```
97 -- QUERY 7: Fields with no sensors installed
98 -- Used in: Technical Dashboard
99 SELECT
100     fi.field_id,
101     fi.field_name,
102     fi.soil_type,
103     fi.area_hectares,
104     fa.farm_name,
105     u.full_name AS owner_name
106 FROM fields fi
107 JOIN farms fa ON fi.farm_id = fa.farm_id
108 JOIN users u ON fa.owner_id = u.user_id
109 WHERE fi.field_id NOT IN (
110     SELECT DISTINCT field_id FROM sensors
111 )
112 ORDER BY fa.farm_name;
```

Explanation: This query retrieves all fields that do not have any sensors installed. It displays information such as field id, name, soil type, area size, farm name and the owner's name. The query compares the field tables with sensor table and selects only fields whose IDs are not found in the sensors records.

Output:

field_id	field_name	soil_type	area_hectares	farm_name	owner_name
6	Greenhouse Section	silty	17.00	Papadopoulou Family Farm	Maria Papadopoulou

Query: Fields which generate most alerts

```
SELECT
    f.field_name,
    COUNT(a.alert_id) AS total_alerts
FROM fields f
JOIN sensors s ON f.field_id = s.field_id
JOIN sensor_readings sr ON s.sensor_id = sr.sensor_id
JOIN alerts a ON sr.reading_id = a.reading_id
GROUP BY f.field_name
HAVING COUNT(a.alert_id) > (
    SELECT AVG(alert_count)
    FROM (
        SELECT COUNT(*) AS alert_count
        FROM alerts
        GROUP BY reading_id
    )
);
```

Explanation: This query calculates the total number of alerts for each field by joining the fields, sensors, sensor readings, and alerts tables. It then compares these totals with the average alert count and returns only the fields that have a higher-than-average number of alerts.

Output:

field_name	total_alerts
North Grain Field	2

Query: All users with their role information

```
SELECT
    user_id,
    full_name,
    email,
    role,
    created_at
FROM users
ORDER BY role, full_name;
```

Explanation: This query retrieves user information from the users table, including the user ID, full name, email, role, and account creation date. The results are sorted by role and then alphabetically by full name to organize the user records more clearly.

Output:

user_id	full_name	email	role	created_at
13	AbdulAziz Kulfat	22304307@student.ciu.edu.tr	admin	2026-05-03 18:14:12.690692
1	Michael Admin	michael.admin@smartfarm.cy	admin	2026-04-26 17:04:25.727676
12	Anna Economou	anna.eco@gmail.com	customer	2026-04-26 17:04:25.727676
11	Dimitris Vassiliou	dimitris.v@yahoo.com	customer	2026-04-26 17:04:25.727676
10	Fatima Al-Hassan	fatima.h@gmail.com	customer	2026-04-26 17:04:25.727676
9	James Mitchell	james.m@outlook.com	customer	2026-04-26 17:04:25.727676
14	mutaz abuallan	mutaz@gmail.com	customer	2026-05-05 16:51:48.233672
8	Sophie Laurent	sophie.l@gmail.com	customer	2026-04-26 17:04:25.727676
4	Andreas Christodoulou	andreas.c@smartfarm.cy	farmer	2026-04-26 17:04:25.727676
3	Maria Papadopoulou	maria.p@smartfarm.cy	farmer	2026-04-26 17:04:25.727676
15	sara mabrouk	sara@gmail.com	farmer	2026-05-07 07:23:06.385846
2	Yiannis Georgiou	yiannis.g@smartfarm.cy	farmer	2026-04-26 17:04:25.727676

3. Software:

We built a GUI to make our system easier, faster and more user-friendly for all the users. Instead of using complex commands or databases queries, users can interact with the system through visual dashboards, forms, charts and buttons.

a. Connection code:

SUPABASE_URL= <https://xplklnxmxvwrrmdoseam.supabase.co>

SUPABASE_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6InhwbGtsbnhteHZ3cnJtZG9zZWZtliwicm9sZSI6ImFub24iLCJpYXQiOiE3NzcyMTczODMsImV4cCI6MjA5Mjc5MzM4M30.w0wFqnIsFJ1QRpzj6N4a24gL5XdPOt3daNKw2U4m3Vc

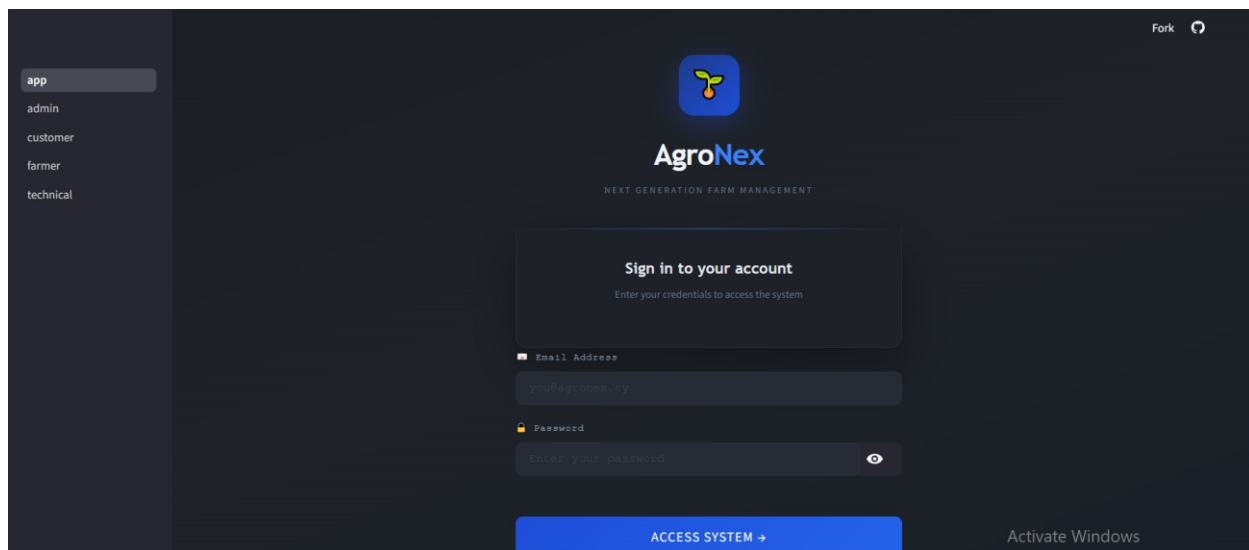
Once the client is created, all database operations are performed through it:

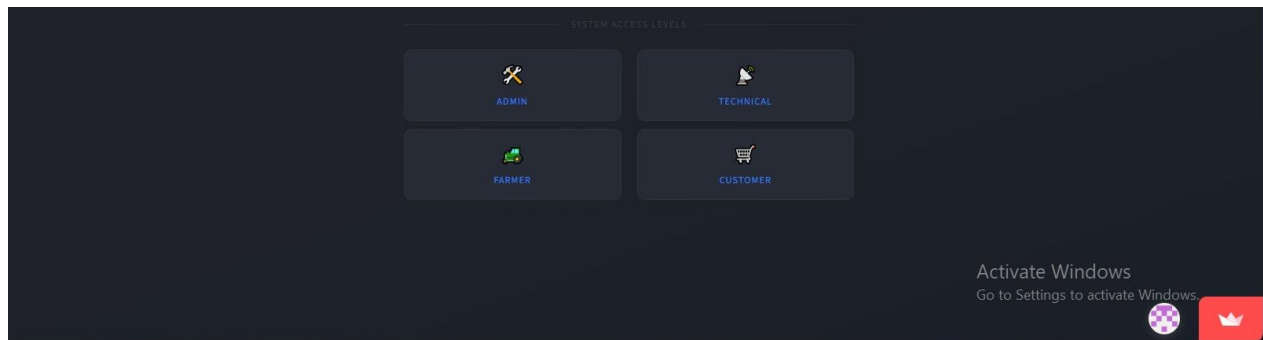
- SELECT → `supabase.table("table_name").select("*").execute()`
- INSERT → `supabase.table("table_name").insert({...}).execute()`
- UPDATE → `supabase.table("table_name").update({...}).eq("id", value).execute()`
- DELETE → `supabase.table("table_name").delete().eq("id", value).execute()`
- Functions & Procedures → `supabase.rpc("function_name", {params}).execute()`

The supabase-py library communicates with the database through Supabase's PostgREST API, which translates each Python method call into a SQL query and executes it securely on the PostgreSQL database hosted on Supabase's cloud servers.

b. Graphical User Interface:

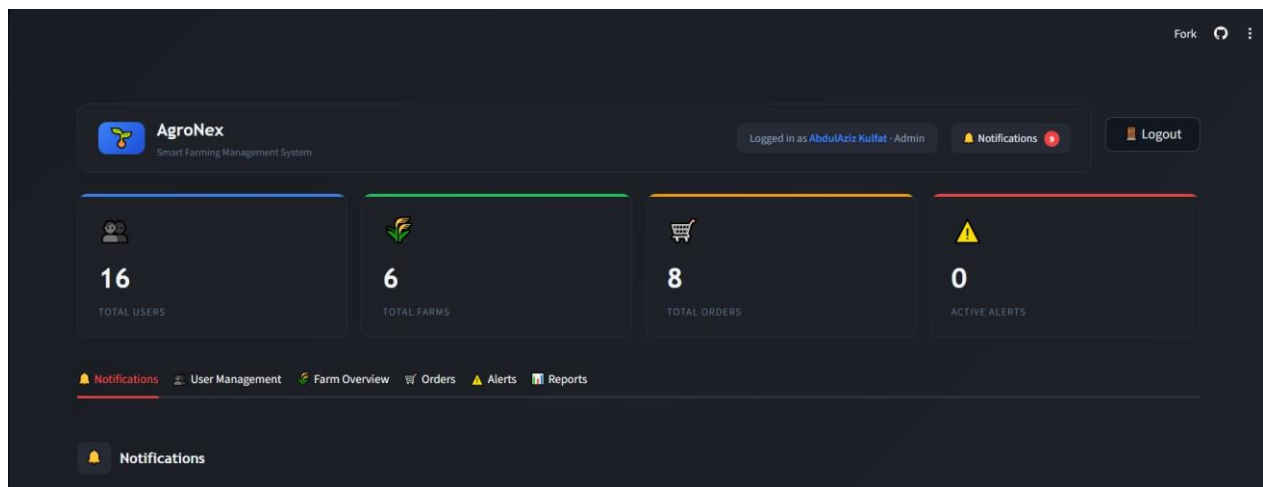
- Login Page:

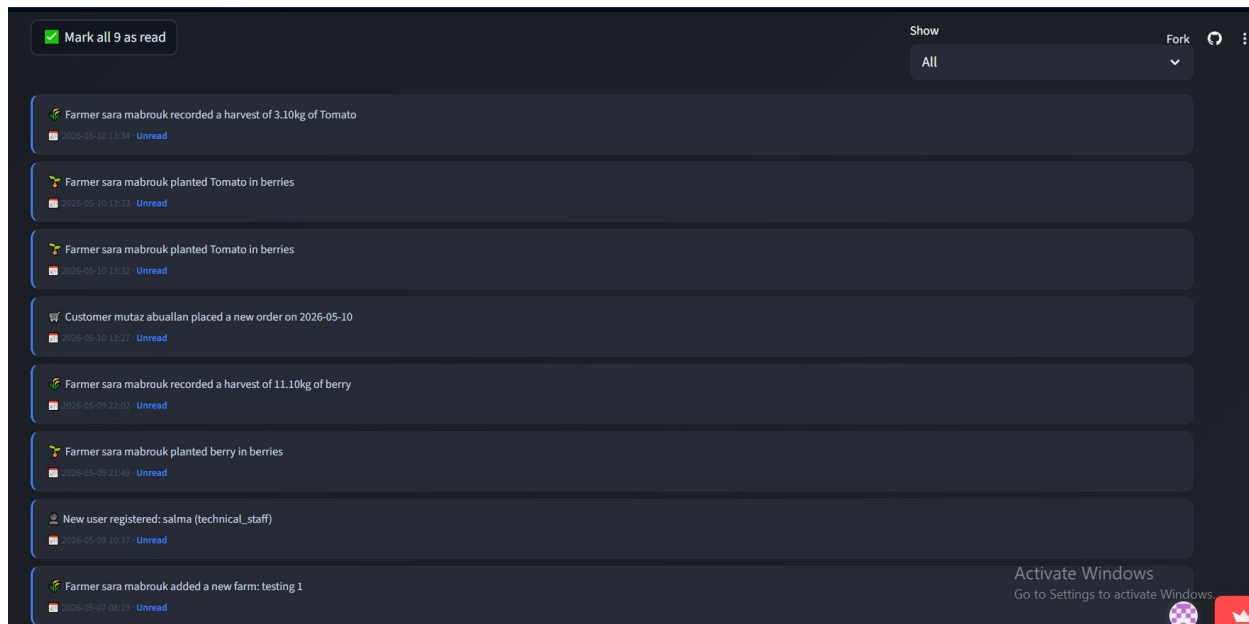




Explanation: This window handles user logging and structural group checks. It acts as a gatekeeper using securely stored credentials to determine if an arriving user holds an 'admin', 'farmer', or 'customer' role, modifying UI dashboard access based on those privileges.

- Admin Page

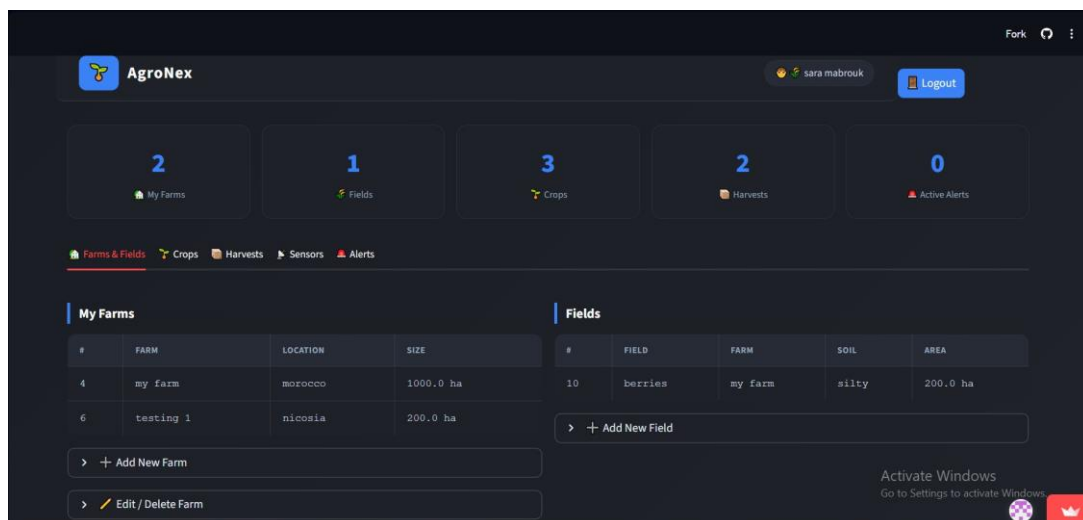




Explanation: This interface shows the AgroNex Admin dashboard and notification console, which gives administrators full control and a clear overview of system activity. At the top, summary cards display key information such as the total number of users, farms, orders, and active alerts, along with a notification badge showing unread system updates.

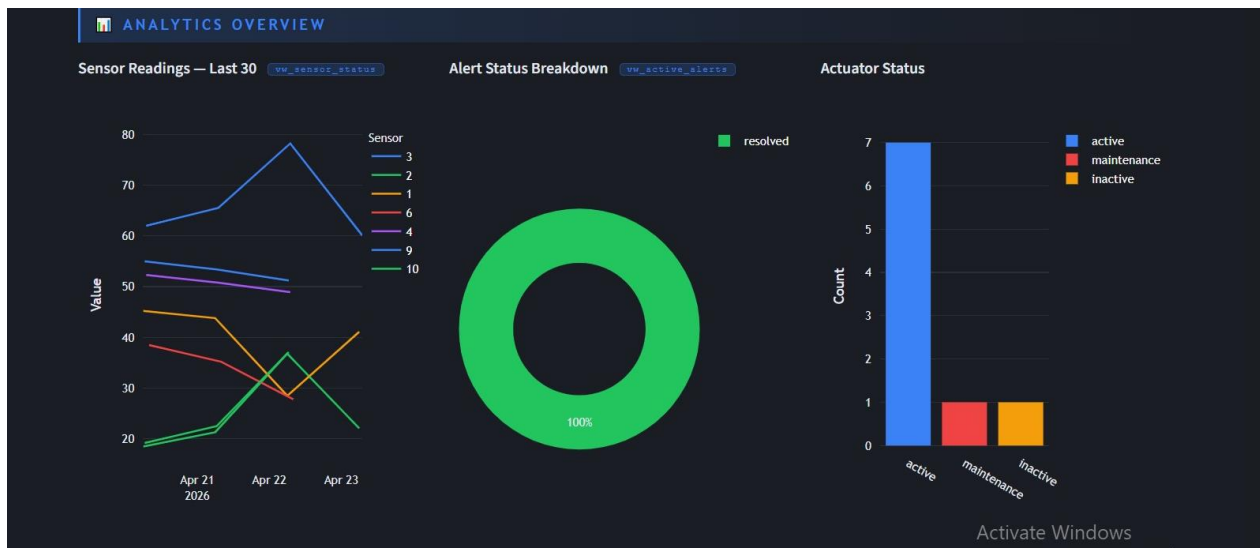
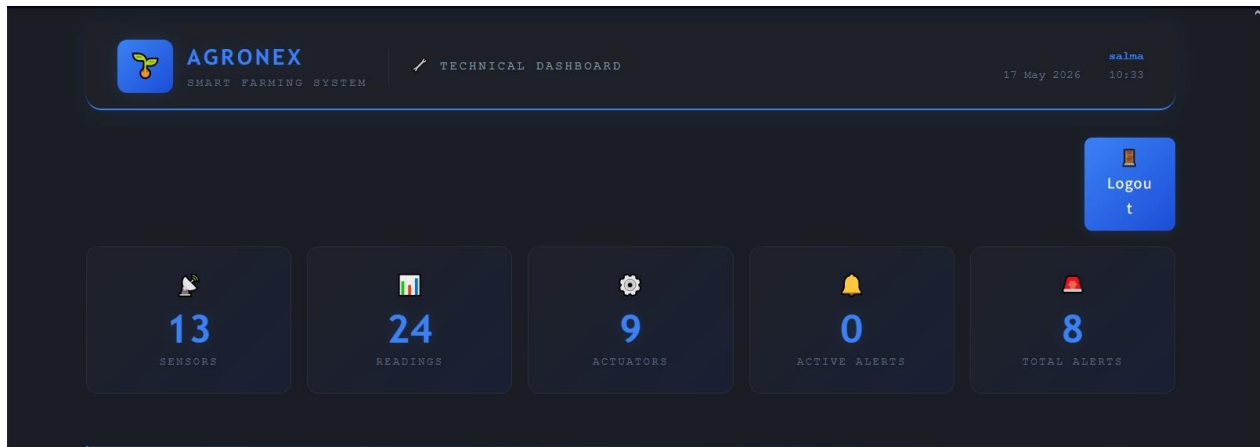
Below the dashboard, the Notifications section works as a live activity feed that automatically records important system events, such as user registrations, crop planting, harvest logging, and new orders.

- Farmer Page:



Explanation: This window is the main dashboard and working area of the AgroNex web application. At the top, dynamic metric cards display real-time information from the cloud database, such as the total number of farms, active fields, crops, harvest records, and environmental alerts. Below the dashboard, there is data tables that show live information from the Farms and Fields sections.

- Technical Staff:



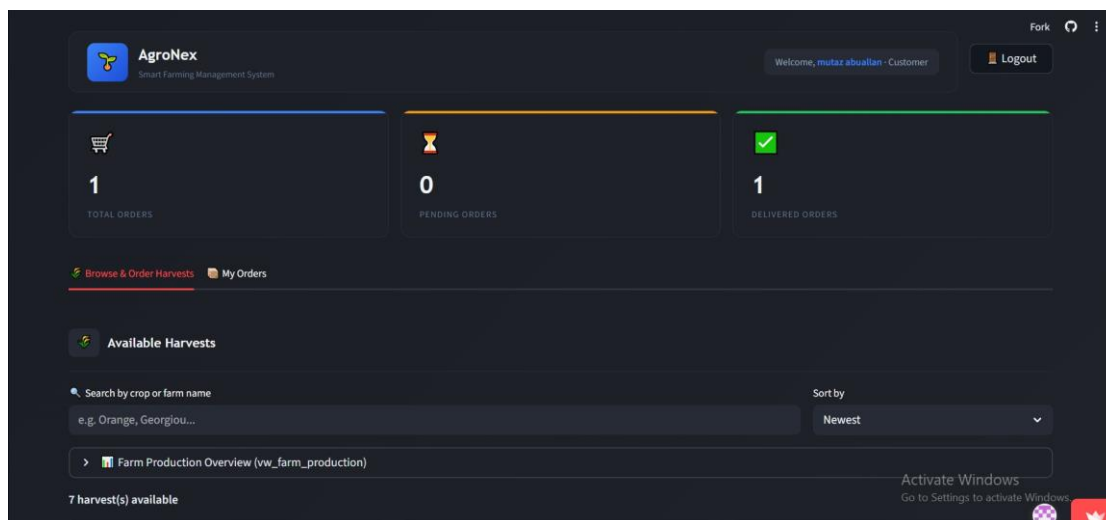
Sensors & Stability							
Sensor Management							
ID	Type	Field	Farm	Installed By	Installed At	Latest Reading	Stability
1	soil_moisture	North Grain Field	Georgiou Organic Farm	Kostas Ioannou	2025-12-01T09:00	41.1 %	MODERATE
2	temperature	North Grain Field	Georgiou Organic Farm	Kostas Ioannou	2025-12-01T09:30	22.1 °C	MODERATE
3	humidity	South Vegetable Plot	Georgiou Organic Farm	Elena Stavrou	2025-12-05T10:00	60.1 %	MODERATE
4	soil_moisture	South Vegetable Plot	Georgiou Organic Farm	Elena Stavrou	2025-12-05T10:30	48.9 %	STABLE
5	temperature	East Orchard Zone	Georgiou Organic Farm	Kostas Ioannou	2025-12-10T08:00	—	NO DATA
6	soil_moisture	Main Citrus Field	Papadopoulou Family Farm	Nikos Petrou	2026-01-05T09:00	27.8 %	MODERATE
7	humidity	Main Citrus Field	Papadopoulou Family Farm	Nikos Petrou	2026-01-05T09:30	—	NO DATA
8	temperature	Vineyard Block A	Papadopoulou Family Farm	Elena Stavrou	2026-01-10T10:00	—	NO DATA
9	soil_moisture	Potato Field Alpha	Christodoulou Agri	Kostas Ioannou	2025-10-15T08:00	51.2 %	STABLE
10	temperature	Potato Field Alpha	Christodoulou Agri	Kostas Ioannou	2025-10-15T08:30	37.1 °C	MODERATE

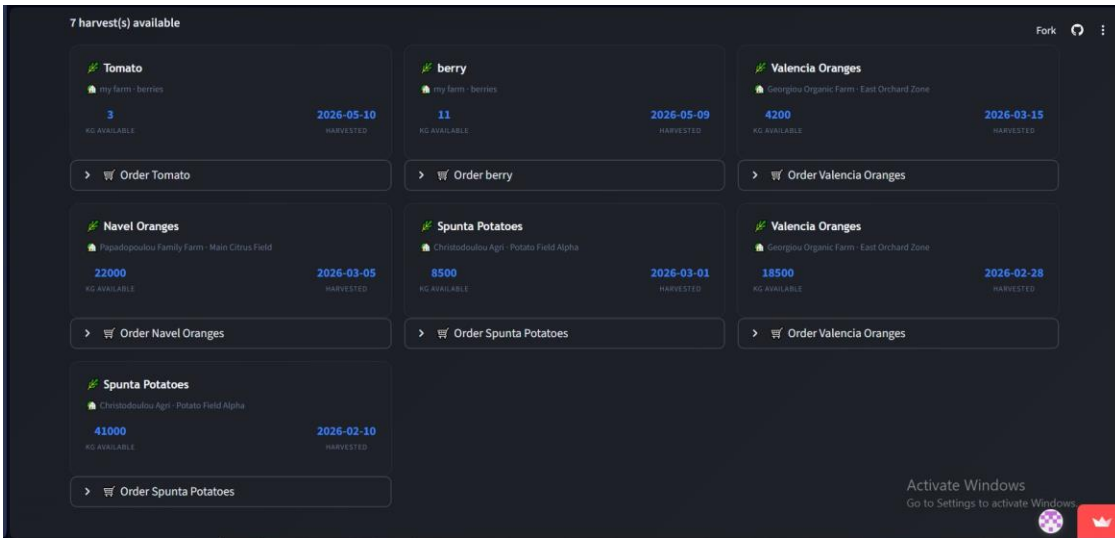
Explanation: The AgroNex Dashboard is designed to help monitor and manage the system's infrastructure easily. At the top, there is cards that show important information like the number of active sections, readings and actuators.

The Analytics Overview section displays charts and graphs to help users understand system performance, including sensor data trends, alert status breakdowns, and device conditions such as active, maintenance, or inactive.

The Sensor Management section provides a detailed table with information about each device, including its location, installation date, assigned technician, and real-time performance status.

- Customer Page:





Explanation: The AgroNex Customer Portal allows customers to browse and order harvest products online. It displays summary cards for total, pending, and delivered orders, along with searchable product listings showing available crops, farm locations, quantities, and harvest dates. Customers can place orders directly using the “+ Order” button.

4. GitHub:

GitHub link: [smart-farming-ddms/.env at main · 22304307AbdulAzizKulfat/smart-farming-ddms](https://github.com/22304307AbdulAzizKulfat/smart-farming-ddms)

Application link: <https://agronex.streamlit.app/>

5. Conclusion:

This project shows the design and development of a Smart Farming system called AgroNex. The system was built using ER Diagram that includes 14 connected tables that manage different farming activities such as farms, fields, crops, harvests, crops, sensors, actuators and customer orders.

The database uses SQL, stored procedures, triggers, functions and views to automate important tasks such as recording harvests, placing orders, generating alerts and sending notifications to administrators.

A user-friendly Python GUI was created to help users interact with the system easily without needing to use database commands directly. The AgroNex system demonstrates how database technologies can help improve farming management, increase efficiency, maintain accurate data and support better decision-making.